Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Command & Code Injection

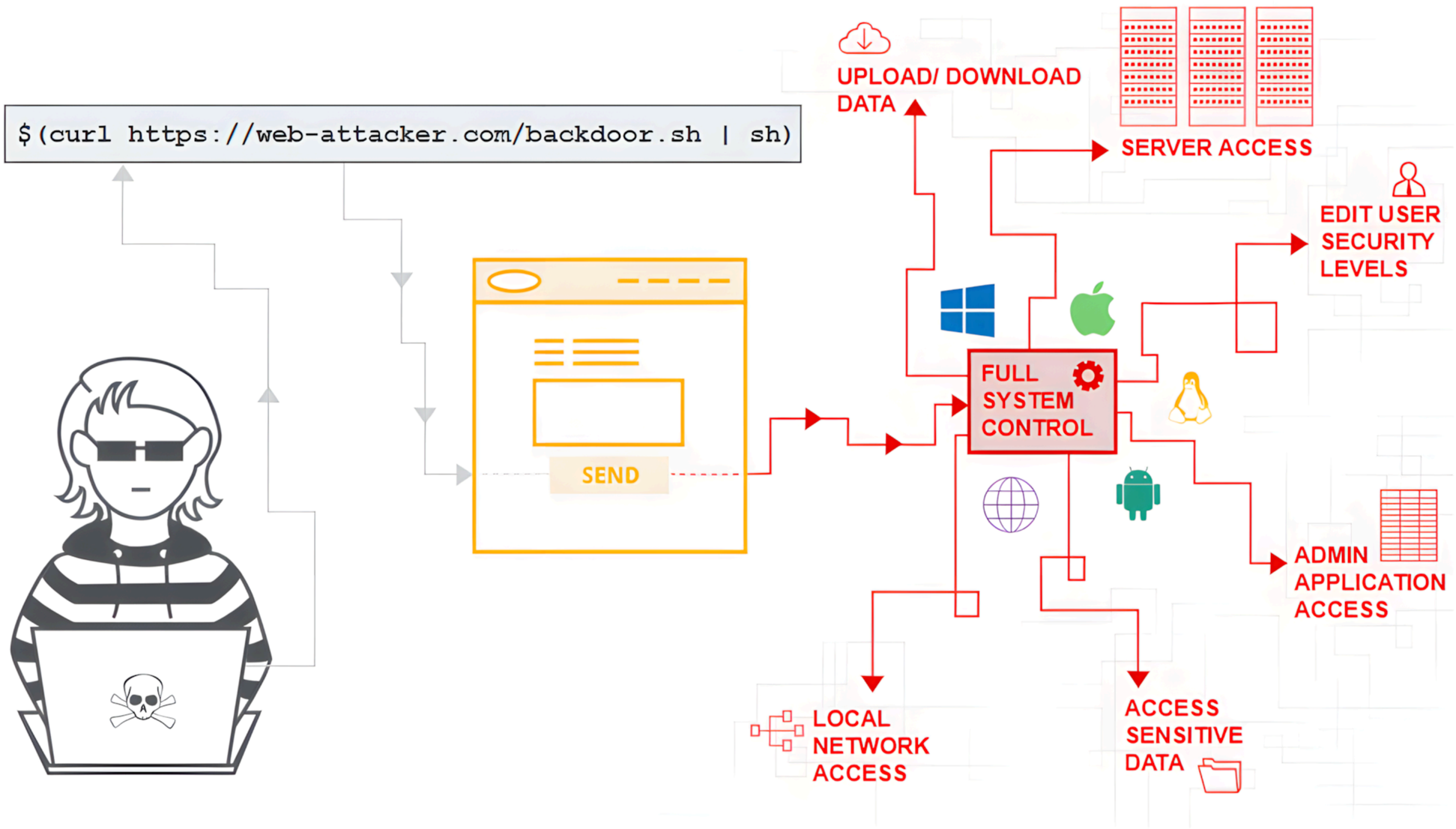
Data Privacy and Security

Data Science and Management (DASMA)

LUISS

Command Injection

Command Injection in a Nutshell



System Commands & the Shell

A **shell** is a program that **interprets and executes system commands**

- Common shells include `cmd.exe` and `PowerShell` on Windows, and `bash/zsh` on Mac
- System commands include managing files and directories, controlling running processes, configuring the network, and interacting with the system at a low level
- You can interact with the shell manually by typing commands in a terminal, or directly from your code

Web applications often rely on **system commands** to **perform OS-level tasks** such as...

- Pinging a host e.g., `ping`
- Processing files e.g., `cat`, `rm`, `cp`
- Navigating the filesystem e.g., `ls`, `find`, `cd`
- ...

Command Injection Attacks

Most programming languages provide functions to execute **system commands**, e.g., `system` in PHP

ping.php

```php
<?php
  system("ping -c 4 " . $_GET["ip"] . " -i 1");
?>
```

- `system` starts a new shell which is used to process the command given as parameter to the function
- The page `ping.php` above uses `system` to ping an IP address provided by the user via the ip variable

Command Injection Attacks

Most programming languages provide functions to execute **system commands**, e.g., `system` in PHP

ping.php

```php
<?php
  system("ping -c 4 " . $_GET["ip"] . " -i 1");
?>
```

- `system` starts a new shell which is used to process the command given as parameter to the function
- The page `ping.php` above uses `system` to ping an IP address provided by the user via the ip variable

> Feeding user input to the function **without validation** can lead to **disasters** :)

Intended Usage

GET /ping.php?ip=8.8.8.8 HTTP/2
Host: example.com

example.com

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=63 time=270 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=63 time=20.0 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=63 time=24.6 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=63 time=30.2 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3039ms
rtt min/avg/max/mdev = 20.048/86.493/270.999/106.585 ms
```

Attack

GET /ping.php?ip=8.8.8.8; cat /etc/passwd # HTTP/2
Host: example.com

example.com

Output of ping

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=63 time=270 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=63 time=20.0 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=63 time=24.6 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=63 time=30.2 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3039ms
rtt min/avg/max/mdev = 20.048/86.493/270.999/106.585 ms
```

Output of cat

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
...
```

How the Attack Works

Payload: `8.8.8.8; cat /etc/passwd #`

- `8.8.8.8` : valid IP address, so the ping command runs normally without raising suspicion
- `;` : separator - tells the shell to **run the next command** regardless of the outcome of the previous one
- `cat /etc/passwd` : reads and prints the contents of the `passwd` file, listing all users on the system
- `#` : **comments out** the rest of the line (`-i 1`) so the command doesn't break

How the Attack Works

Payload: `8.8.8.8; cat /etc/passwd #`

- `8.8.8.8` : valid IP address, so the ping command runs normally without raising suspicion
- `;` : separator - tells the shell to **run the next command** regardless of the outcome of the previous one
- `cat /etc/passwd` : reads and prints the contents of the `passwd` file, listing all users on the system
- `#` : **comments out** the rest of the line (`-i 1`) so the command doesn't break

Result: both `ping 8.8.8.8` and `cat /etc/passwd` are executed, exposing the contents of `passwd`

How to Find Command Injection

Black Box - **no access** to codebase

- Identify areas where the web application **accepts user input** (URLs, form fields, headers)
- **Submit special shell characters**, such as `;`, `|`, &&, and observe how the application reacts
- Look for unexpected errors, delays, or changes in output that suggest the input reached the shell

How to Find Command Injection

Black Box - **no access** to codebase

- Identify areas where the web application **accepts user input** (URLs, form fields, headers)
- **Submit special shell characters**, such as `;`, `|`, &&, and observe how the application reacts
- Look for unexpected errors, delays, or changes in output that suggest the input reached the shell

White Box - **access** to codebase

- Search the codebase for **dangerous functions that execute system commands** (`system()`, `exec()`, in PHP, `os.system()` in Python, ...)
- **Trace how user input flows** into those functions → if it reaches them without sanitization **it's vulnerable**

How to Exploit Command Injection

Once a vulnerable input is identified, the goal is to **inject additional commands** alongside the legitimate one using **command chaining operators**:

- `;` : runs the next command **regardless of the outcome** of the previous one
- `|` : pipes the **output of the first command as input** to the second
- `&&` : runs the next command **only if the previous one succeeded**
- `||` : runs the next command **only if the previous one failed**
- `` ` ` `` : executes the command inside the backticks and **substitutes its output inline**

From there, the attacker can **progressively escalate**:

- Start with **harmless commands** like `whoami` or `id` to confirm execution
- Move on to reading sensitive files, listing directories, **establishing a persistent foothold** on the server

Code Injection

Code Injection Attacks

Most programming languages provide functions to **evaluate and execute code dynamically** e.g., `eval` in PHP

calc.php

```php
<?php
  eval("echo " . $_GET["expr"] . ";");
?>
```

- `eval` takes a string and executes it as code, allowing dynamic expression evaluation
- The page `calc.php` above uses eval to compute an expression provided by the user via the `expr` variable

Code Injection Attacks

Most programming languages provide functions to **evaluate and execute code dynamically** e.g., `eval` in PHP

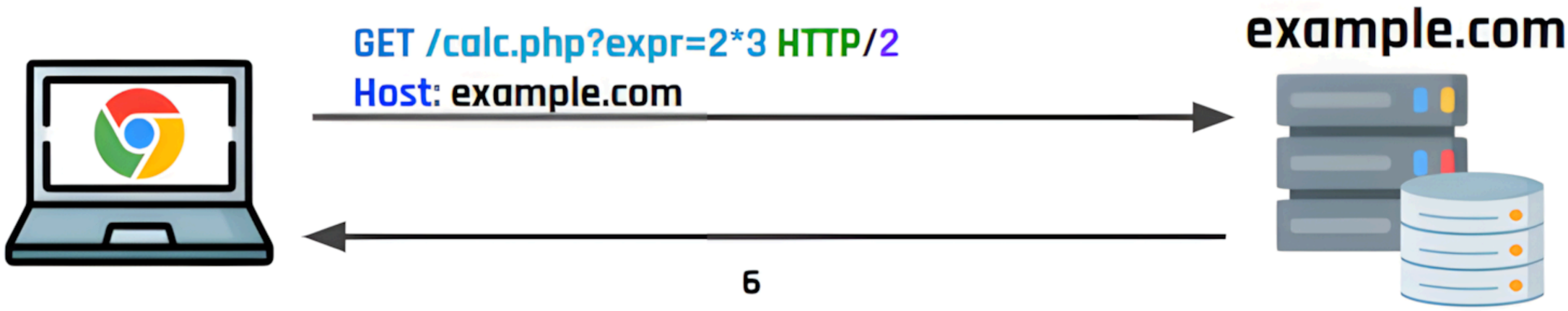
calc.php

```php
<?php
  eval("echo " . $_GET["expr"] . ";");
?>
```

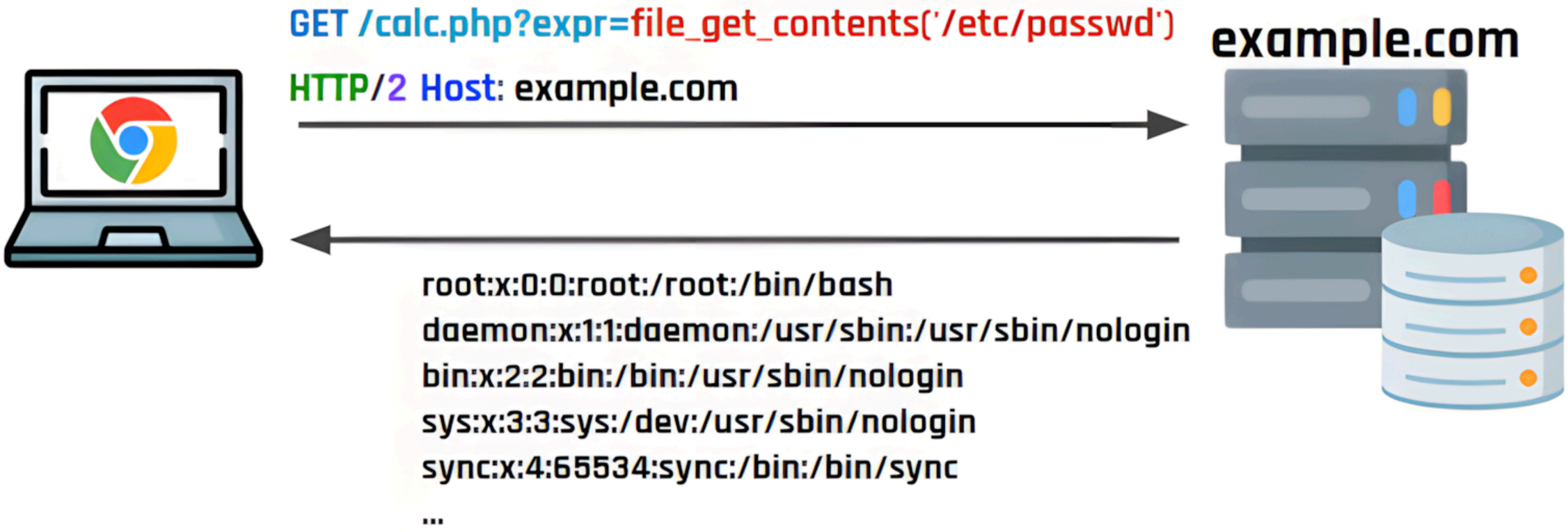
- `eval` takes a string and executes it as code, allowing dynamic expression evaluation
- The page `calc.php` above uses eval to compute an expression provided by the user via the `expr` variable

> Feeding user input to the function **without validation** can lead to **disasters** :)

Intended Usage



Attack


GET /calc.php?expr=file_get_contents('/etc/passwd')
HTTP/2 Host: example.com
example.com
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
...

How the Attack Works

Payload: `file_get_contents('/etc/passwd')`

- `file_get_contents('...')` : native PHP function that reads the contents of a file
- `/etc/passwd` : file to read

Result: `eval` executes the injected function as legitimate code, returning the contents of passwd

How to Find and Exploit Code Injection

Finding code injection follows the same approach as finding command injection:

- Identify areas where the web application **accepts user input**
- Test them with **special characters** such as quotes, parentheses, and semicolons to observe how the application reacts

The key difference lies in **exploitation**: instead of chaining shell commands, the attacker **injects code** in the language the application runs:

- In PHP: `file_get_contents()`, `system()`, `phpinfo()`
- In Python: `os.system()`, `os.popen()`, `exec()`

Command Injection vs Code Injection

Both vulnerabilities stem from the same root cause, **unsanitized user input being passed to a dangerous function**, but they operate at different levels:

- Command Injection **escapes the application** to **interact with the operating system shell**, executing system-level commands
- Code Injection **stays within the application** to **force execution of code** in the same language as the application itself, with access to all its functions and libraries

Types of Command Injection

In-band Command Injection

Consists of an attacker executing commands on the host operating system via a vulnerable application and **receiving the response** of the command in the application.

Blind Command Injection

Consists of an attacker executing commands on the host operating system via a vulnerable application that **does not return the output** from the command within its HTTP response.

- It can still be leveraged via other channels (e.g, out-of-band HTTP requests, time delays)
- Example of time-based payload: `file_exists('/etc/passwd') ? sleep(10) : sleep(0)`

Impact of Command & Code Injection Attacks

- Unauthorized access to the application and host operating system.
  - **Confidentiality** – injection used to view sensitive information.
  - **Integrity** – injection used to alter content in the application.
  - **Availability** – injection used to delete content in the application.
- **Remote code execution** on the operating system
- **Complete takeover** of the server machine

Preventing Code & Command Injection

NEVER use functions like `eval` that dynamically evaluate strings as code (validation is too error prone here)

- Avoid functions that execute system commands; Use **safer alternatives**: several programs come with bindings for different languages
  - Example: use `mkdir()` instead of `system("mkdir /dir_name")`
- If you REALLY want to use such functions, have **strong input validation** in-place e.g., validate against a **whitelist** of permitted values

Reduced privileges of web server

- Use **sandbox environment** (e.g., chroot jail, SELinux, containers) to enforce boundary between web server and the OS

Evading Blacklist Filters

Developers will often use **blacklists** to block known dangerous characters (e.g., ;, |, &&), however blacklists are notoriously hard to get right, and can usually **be bypassed** via:

- URL encoding: `;` becomes `%3B`, `|` becomes `%7C`
- Double encoding: `%3B` becomes `%253B`
- Alternate separators: on Mac/Linux, a newline (`%0A`) can also chain commands
- Whitespace substitution: `$IFS` can replace spaces on Linux when spaces are blacklisted
- Case variation: some filters are case-sensitive, e.g., `PHPinfo()` instead of `phpinfo()`

Useful Resources

- Web Security Academy - OS Command Injection
- **Command Injection Payloads**
- OWASP Command Injection
- OWASP Code Injection
- OWASP OS Command Injection Defense Cheat Sheet
- OWASP Code Injection Defense Cheat Sheet

smartcat3

Web Challenge 02

smartcat3 - Reconnaissance

smartcat3 (500 points) command injection

Damn it, that stupid smart cat litter is broken again. Now only the debug interface is available here and this stupid thing only permits one ping to be sent! I know my contract number is stored somewhere on that interface but I can't find it and this is the only available page! Please have a look and get this info for me!

https://web02.webhack.it

Flag

Submit

What can we observe? The challenge...

- presents a ping debugging interface with a text field accepting a destination input;
- returns the output of the ping command directly in the page.

smartcat3 - Hypothesis

If we try to input a special character, we get:

Smart Cat debugging interface

Ping destination: 8.8.8.8; find

Ping results:

```
Bad character   in dest
```

This could hint at the presence of a blacklist filtering dangerous characters...

PAAS

Web Challenge 05

PAAS - Reconnaissance

PAAS (500 points) web

WebHackIT is proud to announce PAAS - PI as a service!

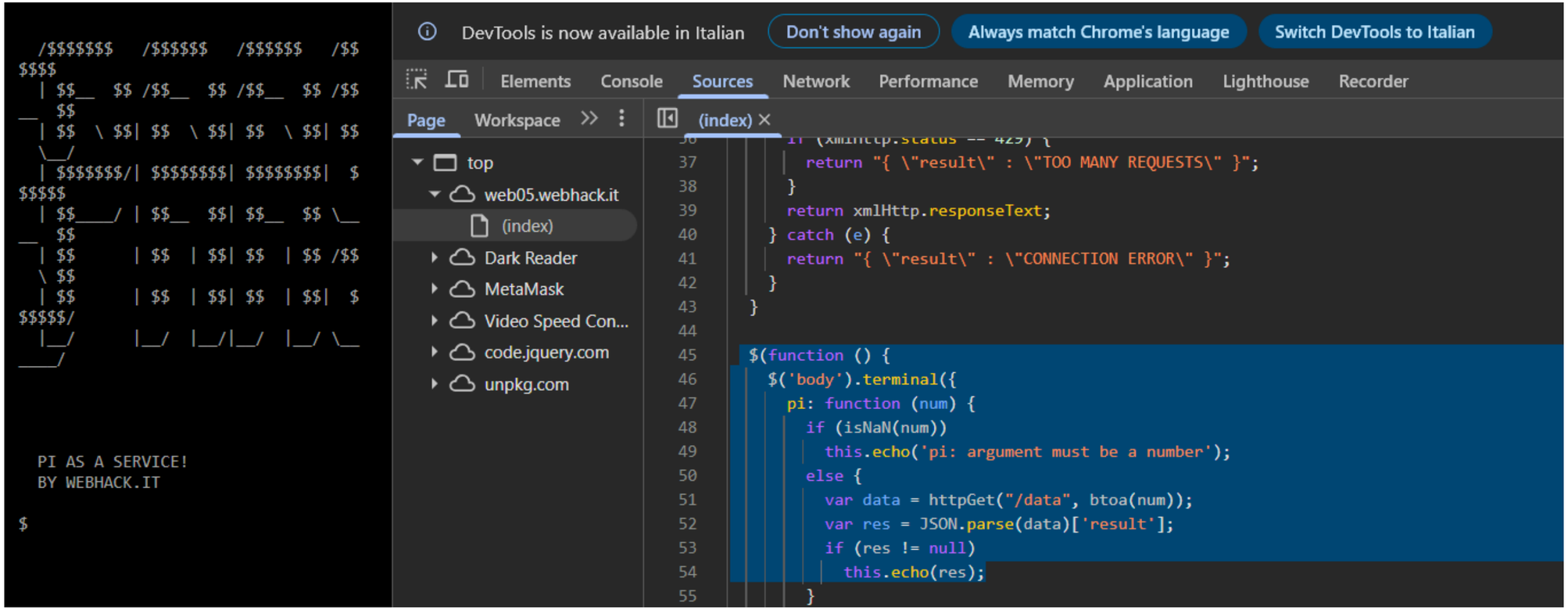
https://web05.webhack.it

Flag

Submit

What can we observe? The challenge...

- presents a terminal-like interface in the browser with a single available command: `pi`;
- the `pi` command accepts a number as argument and returns digits of pi;

PAAS - Hypothesis

Inspecting the page source reveals that the input is encoded with `btoa()` (base64) and sent via POST to a `/data` endpoint - could the server be dynamically evaluating our input...?